

一、簡介

本次作業以 0-1 Knapsack Problem 為主題，實作三種樹搜尋演算法以比較效能：Depth-First Search (DFS)、Breadth-First Search (BFS)、Best-First Search。

二、程式實作

- test_runner.cpp：主程式，負責測試與輸出結果
- dfs_knapsack.cpp：DFS 實作
- bfs_knapsack.cpp：BFS 實作
- best_knapsack.cpp：Best-First Search 實作
- best_knapsack_improved.cpp：Best-First Search 改良版

三、演算法

DFS：遞迴實作，搭配 memoization 避免重複計算，優先探索深層節點。

BFS：queue 實作，完整展開所有層級狀態，保證找到最優解，但效率較低。

Best-First Search：使用 priority queue，實作兩種 bound 計算方式：

版本 A 為簡單估計剩餘物品最大價值，

版本 B 結合當前累積價值，pruning 更嚴格。

四、測試設定

物品數量：10、15、20

物品重量範圍：10 - 100

背包容量設為物品重量總和的一半。

五、 實驗結果與分析

n	Range	Capacity	DFS_MaxVal	DFS_Nodes	BFS_MaxVal	BFS_Nodes	Best-First_MaxVal	Best-First_Nodes	Best-First2_MaxVal	Best-First2_Nodes
10	1-10	21	416	237	416	1,293	236	45	404	120
10	10-100	266	246	677	246	1,365	132	79	238	249
15	1-10	45	526	754	526	40,707	242	309	478	1,493
15	10-100	484	582	4,807	582	39,136	273	273	507	2,439
20	1-10	54	677	1,369	677	1,162,976	363	2,437	655	30,801
20	10-100	547	583	9,923	583	1,198,423	307	367	544	2,230
10	1-10	30	453	318	453	1,212	195	31	369	61
10	10-100	282	434	604	434	1,283	174	15	354	60
15	1-10	35	568	602	568	41,068	302	119	568	707
15	10-100	446	589	4,381	589	37,562	239	65	519	711
20	1-10	63	774	1,646	774	1,207,372	328	345	726	6,361
20	10-100	488	600	9,656	600	1,223,386	252	1,255	540	6,877
10	1-10	19	323	234	323	1,238	174	29	314	67
10	10-100	267	317	718	317	1,202	172	45	305	91
15	1-10	40	599	786	599	36,426	280	215	577	1,243
15	10-100	382	643	4,085	643	40,329	340	65	621	246
20	1-10	58	660	1,607	660	1,207,929	300	815	622	9,833
20	10-100	703	796	12,536	796	1,232,556	372	1,527	730	22,495

節點訪問數比較

- DFS 節點數明顯少於 BFS，說明 memoization 有效減少重複計算。
- BFS 節點數隨物品數和容量增加呈爆炸式成長，導致記憶體消耗巨大且效率低下。
- Best-First Search 節點數遠少於 BFS，改良版 (Best-First2) 節點數較初版顯著減少且解更接近最優，但仍未完全保證找到最佳解。

觀察

- DFS 是目前最穩定且能找到最優解的方法，但在大規模問題仍需較多運算時間。
- BFS 因為狀態空間龐大，實務上限制較多，不適合物品數多的問題。
- Best-First Search 的效能與解的品質受限於 bound 函數的準確度，改良 bound 是提升演算法效能的關鍵。

改進方向

- 持續優化 bound 函數，使剪枝更精準且不漏解。
- 結合動態規劃 (DP) 方法處理特殊情況，提高整體效率和正確性。
- 調整資料結構或剪枝策略，平衡搜尋深度與節點數。

六、 結論

bound 優化

目前的 bound 函數是用物品的 value-to-weight ratio 做簡單估計，這會造成剪枝過多，容易錯過真正的最佳解。未來優化方向：

- 先對物品按價值密度排序，讓估計更有依據。
- 採用更精確的上下界 (upper/lower bound) 估計方法，例如結合動態規劃 (dynamic programming) 結果，提升 bound 的準確度。
- 調整剪枝條件，避免過度剪枝，平衡效率與解的品質。
- 透過實驗調整參數，找到最適合不同問題規模的剪枝策略。

這次實驗比較了三種搜尋演算法在 0-1 背包問題上的表現。結果顯示，DFS 搭配 memoization 能有效減少重複計算，既準確又相對高效，是目前最實用的解法。BFS 雖然能找到最優解，但因節點數爆炸，計算時間和記憶體消耗過大，實際應用受限。Best-First Search 節點數少，節省運算資源，但現有的 bound 函數估計不夠準確，導致有時無法找到最優解。未來改進方向是優化 bound 函數，提高剪枝精度，使 Best-First Search 能更快且準確地求解問題，並針對特殊情況設計更有效率的演算法。